Decision Tree — Complete Notes (English)

Nov 8, 2026 · @Razeen

A Decision Tree is a supervised learning algorithm that builds an **if-else rule tree** automatically from data. It is interpretable, handles non-linear data, and requires no feature scaling.

1. Introduction to Tree-Based Methods

Tree-based methods are a family of supervised learning algorithms that make predictions using a hierarchy of **if-else rules**, learned automatically from training data.

A Simple Example

Predicting whether to play cricket today:

- Is it raining? → Yes → NO
- Is it sunny? → Yes → YES
- Are there enough players? → No → NO

This exact branching logic is what a Decision Tree encodes from data.

Why Use Decision Trees?

Feature	Linear Models	Decision Tree
Non-linear data	Struggles	Handles naturally
Feature scaling required	Yes	No
Interpretable	Partial	Highly interpretable
Handles missing values	Difficult	Yes
Handles categorical data	Requires encoding	Yes

The Tree-Based Family

- Decision Tree — single tree, interpretable baseline
- Random Forest — ensemble of 100+ trees, reduced variance
- Gradient Boosting — trees built sequentially on errors
- XGBoost / LightGBM — optimized boosting, production-grade

Real-World Use Cases: Loan approval (banking), disease diagnosis (healthcare), customer churn prediction (e-commerce), employee attrition (HR).

2. History and Terminology

Algorithm Timeline

Year	Person / Paper	Contribution
1963	Morgan & Sonquist	AID — first tree-based method
1979	Ross Quinlan	ID3 — introduced Information Gain
1984	Breiman et al.	CART — introduced Gini impurity
1993	Ross Quinlan	C4.5 — improved ID3 with pruning
2001	Leo Breiman	Random Forest — ensemble of trees

Scikit-learn implements the **CART algorithm** (Classification and Regression Trees).

Key Terminology

Term	Definition
Root Node	The first split; the most informative feature
Internal Node	An intermediate node that tests a condition
Leaf Node	A terminal node that holds the prediction
Branch / Edge	The connection between two nodes
Depth	Number of levels from root to a leaf
Pure Node	Contains only one class (Gini = 0)
Impure Node	Contains mixed classes (Gini > 0)
Split	Dividing a node into two child nodes
Pruning	Removing branches to prevent overfitting
Parent Node	The node from which a split originates
Child Node	The nodes created after a split

3. Gini Impurity

Gini Impurity measures how **mixed** (impure) a node is. A pure node contains only one class; an impure node contains a mix.

- Pure node: Gini = 0
- Maximally impure (binary, 50-50 split): Gini = 0.5

Formula

```
\text{Gini}(node) = 1 - \sum_{i=1}^{C} p_i^2
```

Where p_i is the proportion of samples belonging to class i.

Worked Examples

Node Contents	p(Yes)	p(No)	Calculation	Gini
[Y, Y, Y, Y]	1.0	0.0	$1 - (1.0^2 + 0.0^2)$	0.0 (Pure)
[Y, Y, Y, N]	0.75	0.25	$1 - (0.75^2 + 0.25^2)$	0.375

Node Contents	p(Yes)	p(No)	Calculation	Gini
[Y, Y, N, N]	0.5	0.5	$1 - (0.5^2 + 0.5^2)$	0.5 (Max Impure)

Gini vs Entropy Comparison

	Gini	Entropy
Formula	$1 - \Sigma\, p^2$	$-\Sigma\, p \log_2(p)$
Range	[0, 0.5]	[0, 1]
Computation	Faster (no log)	Slower
Scikit-learn default	Yes (`criterion='gini'`)	Optional (`criterion='entropy'`)
Results in practice	Usually similar	Usually similar

Intuition: Think of a classroom. If all students prefer the same subject, the class is pure (Gini = 0). If students are split across many subjects, it is impure. The tree algorithm always chooses the split that reduces Gini the most.

4. Tree Construction with Gini Impurity

The CART Algorithm

1. Start with all training samples in the root node
2. For every feature, try every possible split threshold
3. Compute the Weighted Gini for each split
4. Select the split with the highest Gini Gain
5. Create two child nodes
6. Repeat recursively until a stopping criterion is met

Weighted Gini Formula

```
\text{Weighted Gini} = \frac{n_{left}}{n} \times Gini_{left} +
\frac{n_{right}}{n} \times Gini_{right}
```

Gini Gain

```
\text{Gini Gain} = Gini_{parent} - \text{Weighted Gini}_{split}
```

The split with the **highest Gini Gain** is selected at each node.

Numerical Example

10 customers: 6 took a loan (Yes), 4 did not (No). Parent Gini = $1 - (0.6^2 + 0.4^2) = 0.48$.

Candidate Split	Left Node	Right Node	Weighted Gini	Gini Gain
Age > 30	Gini=0.48 (n=5)	Gini=0.32 (n=5)	0.40	0.08
Income > 5L	Gini=0.375 (n=4)	Gini=0.278 (n=6)	0.317	0.163

Income > 5L wins (higher Gini Gain) → selected as the root split.

Stopping Criteria

Parameter	Description
max_depth	Maximum depth of the tree
min_samples_split	Minimum samples required to split a node
min_samples_leaf	Minimum samples required in a leaf node
ccp_alpha	Cost-Complexity Pruning threshold

Any stopping criterion being met prevents further splitting at that node.

5. Implementation with Scikit-learn

Old vs New Approach

	Manual (Scratch)	Scikit-learn
Gini calculation	Manual loops	Automatic
Splitting logic	Manual recursion	Automatic
Pruning	Manual	max_depth, min_samples_*
Visualization	Custom code	plot_tree(), export_text()
Production use	No	Yes

Basic Usage

```
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.model_selection import train_test_split, cross_val_score

# Build the model
dt = DecisionTreeClassifier(
    criterion='gini',       # 'gini' or 'entropy'
    max_depth=5,            # maximum tree depth
    min_samples_split=2,    # minimum samples to split a node
    min_samples_leaf=1,     # minimum samples in a leaf
    random_state=42
)
dt.fit(X_train, y_train)

# Predict
y_pred = dt.predict(X_test)

# Visualize the tree
plot_tree(dt, feature_names=feature_names,
          class_names=class_names, filled=True)

# Extract text rules
print(export_text(dt, feature_names=feature_names))
```

Key Parameters

Parameter	Default	Purpose
criterion	'gini'	Impurity measure used for splits
max_depth	None	Maximum depth of the tree
min_samples_split	2	Minimum samples to allow a split
min_samples_leaf	1	Minimum samples in any leaf
max_features	None	Number of features to consider per split
random_state	None	Seed for reproducibility
ccp_alpha	0.0	Cost-Complexity Pruning threshold

Useful Methods and Attributes

- `dt.get_depth()` — actual depth of the fitted tree
- `dt.get_n_leaves()` — number of leaf nodes
- `dt.feature_importances_` — Gini-based importance per feature
- `dt.score(X_test, y_test)` — accuracy on a dataset
- `dt.predict_proba(X)` — predicted class probabilities

6. Overfitting and Pruning

The Problem

An unconstrained tree memorizes the entire training set (Train accuracy = 100%) but generalizes poorly to new data — this is overfitting.

Bias-Variance Tradeoff in Trees

max_depth	Behavior	Issue
Too small (1–2)	Underfitting	High bias, low variance
Sweet spot (3–7)	Well-balanced	Good generalization
Too large (None)	Overfitting	Low bias, high variance

Finding the Sweet Spot

```
depths = range(1, 20)
cv_scores = []
for d in depths:
    dt = DecisionTreeClassifier(max_depth=d, random_state=42)
    cv = cross_val_score(dt, X, y, cv=5).mean()
    cv_scores.append(cv)

best_depth = depths[np.argmax(cv_scores)]
print(f"Best depth: {best_depth}")
```

Types of Pruning

- **Pre-pruning (Early Stopping):** Stop growing the tree during training — `max_depth`, `min_samples_split`, `min_samples_leaf`
- **Post-pruning:** Grow the full tree, then remove unnecessary branches — `ccp_alpha` (Cost-Complexity Pruning)

Automated Hyperparameter Search

```
param_grid = {
    'max_depth': [3, 5, 7, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 5],
    'criterion': ['gini', 'entropy']
}
gs = GridSearchCV(DecisionTreeClassifier(random_state=42),
                  param_grid, cv=5, scoring='accuracy', n_jobs=-1)
gs.fit(X_train, y_train)
print(gs.best_params_)
```

Rule of Thumb: Always evaluate using `cross_val_score`, not just test accuracy, to get a reliable estimate of generalization performance.

7. Feature Importance and Interpretation

What Is Feature Importance?

Each feature receives a score equal to its total Gini reduction contribution across all splits in the tree. Features at the root or higher levels typically score higher.

```
importances = pd.Series(
    dt.feature_importances_,
    index=feature_names
).sort_values(ascending=False)
print(importances)

# Bar chart
importances.plot(kind='barh', title='Feature Importance')
```

Extracting Human-Readable Rules

The biggest advantage of Decision Trees is their interpretability. Rules can be explained to anyone:

```
export_text(dt, feature_names=features)
# Output:
# |--- petal length <= 2.45
# |   |--- class: setosa
# |--- petal length > 2.45
# |   |--- petal width <= 1.75
# |   |   |--- class: versicolor
# |   |--- petal width > 1.75
# |   |   |--- class: virginica
```

These rules can be handed directly to a doctor, loan officer, or HR manager.

Explaining a Single Prediction

```
patient = X_test.iloc[0]
pred = dt.predict([patient])[0]
prob = dt.predict_proba([patient])[0]
print(f"Prediction: {pred}, P(class 1) = {prob[1]:.3f}")
# Trace the decision path:
node_indicator = dt.decision_path([patient])
```

False Negatives in Medical Contexts

- False Negative (missed disease) = dangerous: patient receives no treatment
- False Positive (healthy flagged) = costly but recoverable

In high-stakes domains, monitor **Recall** (Sensitivity) from `classification_report`, not just overall accuracy.

8. Summary and Next Steps

Complete Topic Overview

Topic	Key Takeaway
Tree-based Methods	Ideal for non-linear data; highly interpretable
Gini Impurity	Measures node purity (0 = pure, 0.5 = maximally impure, binary)
Tree Construction	Greedy algorithm — selects the split with the highest Gini Gain at each node

Topic	Key Takeaway
Implementation	DecisionTreeClassifier; max_depth is the most important parameter
Overfitting	Deep trees overfit; find the sweet spot using cross-validation
Feature Importance	Reflects total Gini reduction; root-level features score highest
Interpretation	export_text() produces human-readable rules for any audience

Limitations of a Single Decision Tree

- High variance: small changes in data can produce a very different tree
- Sensitive to outliers and class imbalance
- Axis-aligned splits only: cannot capture diagonal boundaries

The Road Ahead

Algorithm	How It Improves on a Single Tree	When to Use
Random Forest	Averages 100+ trees — reduces variance	General-purpose strong baseline
Gradient Boosting	Builds trees sequentially on residuals	Higher accuracy needed
XGBoost / LightGBM	Optimized boosting with regularization	Production, competitions

A Decision Tree is the right starting point for any classification problem: it delivers an interpretable baseline quickly. If accuracy is insufficient, move to Random Forest. For production tabular data, XGBoost or LightGBM typically performs best.